Software Engineering

Assignment #3

Project: Smart Campus Room Availability Checker

Zhumash Assylbek

24B031801

Class time;Th,9-12

AI Tools & Models Used: Claude (Sonnet 5.5) and Bolt.new

Bolt link :https://bolt.new/p/72188768

Task 1: Baseline and Plan

1. Scope and Acceptance Criteria (in Own Words)

User Story & Scope: As a student, I want to verify whether a specific campus study room is available for a chosen time interval so that I can select a valid booking slot. The prototype is a single responsive web page for one room using fictional booking data. It strictly checks availability only and never creates, confirms, or saves bookings. Login, databases, backend servers, external APIs, and payment systems are completely excluded.

AC1 (Time Representation & Bounds): Candidate start and end times are integer minutes from midnight (0 <= start < end <= 1440), and start must be strictly greater than now (where now is a valid integer in 0..1439).

AC2 (Maximum Duration): The requested duration (end - start) must not exceed 120 minutes; a duration of exactly 120 minutes is allowed.

AC3 (Room Blocked State): Every request must be rejected if the room is currently marked as blocked (blocked == True).

AC4 (No Overlap with Existing Bookings): Any request that overlaps with an interval (s, e) in existing must be rejected, whereas intervals that merely touch at an endpoint (end == s or start == e) do not overlap and are permitted.

AC5 (Unambiguous Output & Immutability): The checker must display a clear Available / Unavailable result and must never mutate the existing bookings list.

AC6 (Validation Feedback & Mobile Usability): The interface must provide clear validation messages explaining why a request failed and maintain a clean, usable layout on narrow/mobile viewports.

2. Likely Failure Points

1. **Endpoint Touching Error (AC4):** Using <= instead of < in the interval overlap check (start < e and s < end), which would falsely reject valid adjacent bookings such as 660–720 when 600–660 is booked.

2. **Current Time Boundary Error (AC1):** Allowing a booking that starts exactly at now (start == now, e.g., 540–570 when now = 540) instead of enforcing strict inequality start > now.

3. **Python Type & Malformed Input Pitfalls (AC1 & AC5):** In Python, bool is a subclass of int (isinstance(True, int) evaluates to True), and malformed entries inside existing (such as None, single-element tuples (600,), or boolean tuples (True, 5)) can cause unhandled TypeError/ValueError exceptions or silent miscalculations instead of returning False.

3. Function Contract

Signature: can_book(start, end, now, blocked, existing) -> bool

Inputs: start: int, end: int, now: int (minutes after midnight); blocked: bool; existing: Iterable[tuple[int, int]].

Output & Postconditions: Returns True if and only if AC1–AC4 are satisfied; returns False for all invalid requests or malformed inputs without raising unhandled exceptions and without modifying existing (AC5).

4. Initial Assumptions and Manual Verification Plan

Assumptions: All booking intervals lie within a single 24-hour day (0..1440), now is within 0..1439, and all room schedule data is fictional and stored in memory.

Manual Verification: Manually inspect every conditional branch in booking.py, run automated unittest cases in test_booking.py, and manually test all boundary inputs and mobile viewport responsiveness (375px) in the Bolt.new preview.

Task 2: AI-Assisted Python Implementation (booking.py & test_booking.py)

The function can_book(start, end, now, blocked, existing) was implemented in Python 3 using only the standard library.

Branch-by-Branch Review Against AC1–AC5:

1. **Type Validation (_is_int and blocked check - AC1, AC3):** Checks type(value) is int for start, end, and now (explicitly rejecting bool and float) and verifies type(blocked) is bool. If blocked is True, the branch immediately returns False (**AC3**).

2. **Time Range & Current Time Check (AC1):** Verifies 0 <= now <= 1439, 0 <= start < end <= 1440, and start > now. Requests starting at or before now or having start >= end return False (**AC1**).

3. **Duration Limit Check (AC2):** Evaluates if end - start > 120: return False, allowing up to 120 minutes (720–840) and rejecting 121+ minutes (720–841) (**AC2**).

4. **Interval Validation & Overlap Loop (AC4, AC5):** In **v1**, unpacking for s, e in existing: directly raised TypeError/ValueError on malformed existing inputs and accepted [(True, 5)] as (1, 5). In **v2**, iter(existing) is wrapped in try/except TypeError and each entry is validated via _is_valid_interval(item) (requiring a 2-element tuple/list of strict int values). Overlap is checked via if start < e and s < end: return False, which allows

touching endpoints (600–660 and 660–720) (**AC4**) while scanning read-only without mutating existing (**AC5**).

Task 3: Web Prototype with Bolt.new

The single-page web prototype was generated in **Bolt.new** by providing AC1–AC6 and explicitly forbidding login, database, and backend features.

Main Generated Files & Component Roles:

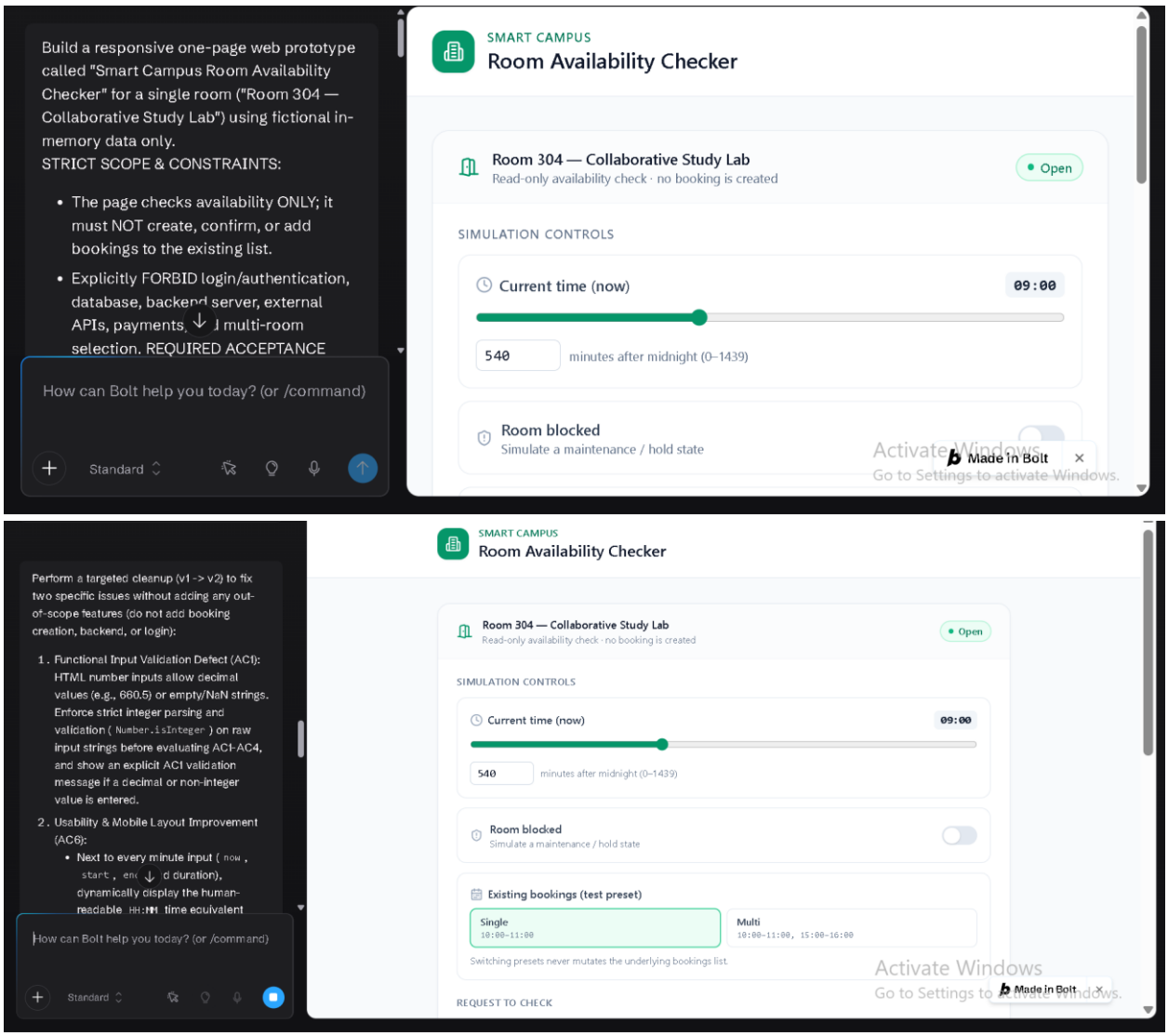
index.html - Root HTML document defining the page title and responsive viewport meta tag.

package.json - Project configuration listing React, TypeScript, Lucide icons, and Tailwind CSS dependencies run by the Vite dev server.

src/main.tsx & src/index.css — Application entry point and responsive styling rules for desktop and mobile layouts.

src/App.tsx - Main single-page component implementing the room status controls (now = 540, blocked = False), read-only display of existing = [(600, 660)], candidate start/end inputs, pure availability validation logic (AC1–AC5), human-readable HH:MM labels, and rule-specific validation feedback (AC6).

Prototype Screenshots (Before & After Cleanup — v1 vs. v2):



Task 4: Traceability Table (Test Execution & Evidence)

Base test configuration: now = 540, blocked = False, existing = [(600, 660)] unless noted otherwise. All 50 unit tests in test_booking.py and all manual browser tests were executed.

AC ID	Implementation / File	Test Input	Expected	v1 Actual	Final Actual (v2)
AC4	booking.py	660-720 (touching endpoint)	True	True	True
AC4	booking.py	630-690 (overlap)	False	False	False
AC3	booking.py	660-720, blocked=True	False	False	False

AC ID	Implementation / File	Test Input	Expected	v1 Actual	Final Actual (v2)
AC2	booking.py	720-840 (exactly 120 min)	True	True	True
AC2	booking.py	720-841 (121 min)	False	False	False
AC1	booking.py	540-570 (starts exactly at now)	False	False	False
AC1	booking.py	700-680 (reversed times)	False	False	False
AC1	booking.py	(-10, 30) and (1380, 1441) (day bounds)	False	False	False
AC1	booking.py	can_book(True, 5, 0, False, []) (bool input)	False	False	False
AC5	booking.py	existing=[(900,960),(600,660),(700,760)]	List unchanged, same order	Unchanged	Unchanged
AC5	booking.py	existing=[(600,)] or None (malformed)	False, no exception	Raises ValueError / TypeError	False
AC5	booking.py	existing=[(True, 5)], request 660-720	False	True (treated as 1-5)	False

AC ID	Implementation / File	Test Input	Expected	v1 Actual	Final Actual (v2)
AC4	Bolt.new prototype	660-720 (touching endpoint)	Available	Available	Available
AC4	Bolt.new prototype	630-690 (overlap)	Unavailable (overlap)	Unavailable	Unavailable
AC3	Bolt.new prototype	660-720, blocked=True	Unavailable (room blocked)	Unavailable	Unavailable
AC2	Bolt.new prototype	720-840 (120 min)	Available	Available	Available
AC2	Bolt.new prototype	720-841 (121 min)	Unavailable (> 120 min)	Unavailable	Unavailable
AC1	Bolt.new prototype	540-570 (start == now)	Unavailable (start <= now)	Unavailable	Unavailable
AC1	Bolt.new prototype	700-680 and bounds -10 / 1441	Unavailable (invalid bounds)	Unavailable	Unavailable
AC1	Bolt.new prototype	630.5 or empty string input	Rejected with AC1	Allowed decimal input in	Rejected with explicit

AC ID	Implementation / File	Test Input	Expected	v1 Actual	Final Actual (v2)
			integer message	HTML field	integer error
AC5	Bolt.new prototype	Repeated availability checks	existing list unchanged	Unchanged	Unchanged
AC6	Bolt.new prototype	Enter 660 & test at 375px mobile width	Clear feedback, HH:MM hint, usable mobile UI	Raw minutes only, dense mobile controls	Shows 11:00 (HH:MM), clean mobile layout & presets

Task 5: Issue Log (Cleanup and Retest)

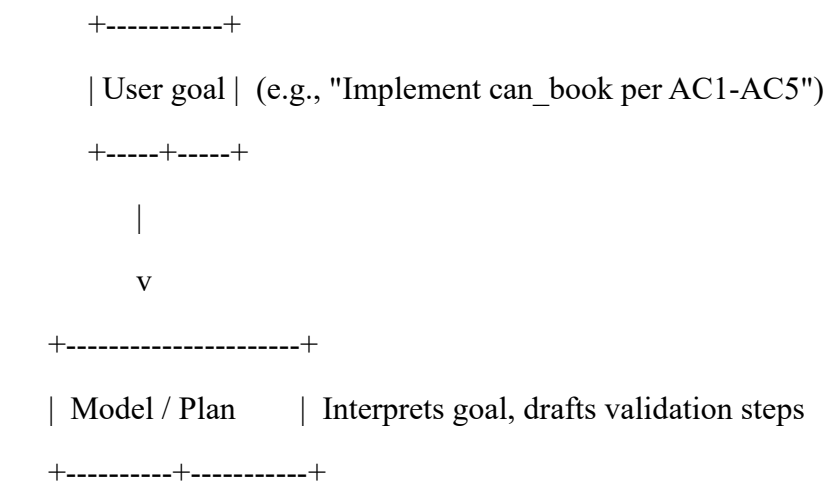
Issue	Reproduction	Priority	Prompt / Change	Retest Result
1. booking.py (Functional / AC5): Unhandled exceptions & boolean elements inside existing	Calling can_book(660, 720, 540, False, [(True, 5)]) in v1 returned True (treating True as integer 1), while existing=None or [(600,)] raised TypeError/ValueError instead of returning False.	High (Reliability & AC5 contract)	Prompted Claude (Stage C/D) to audit v1 for boolean and malformed existing inputs. Added _is_valid_interval(item) using type(x) is int and wrapped iter(existing) in try/except TypeError to fail closed (False).	All 50/50 unit tests pass (including 10 new tests in TestMalformedExisting). existing remains unmutated.

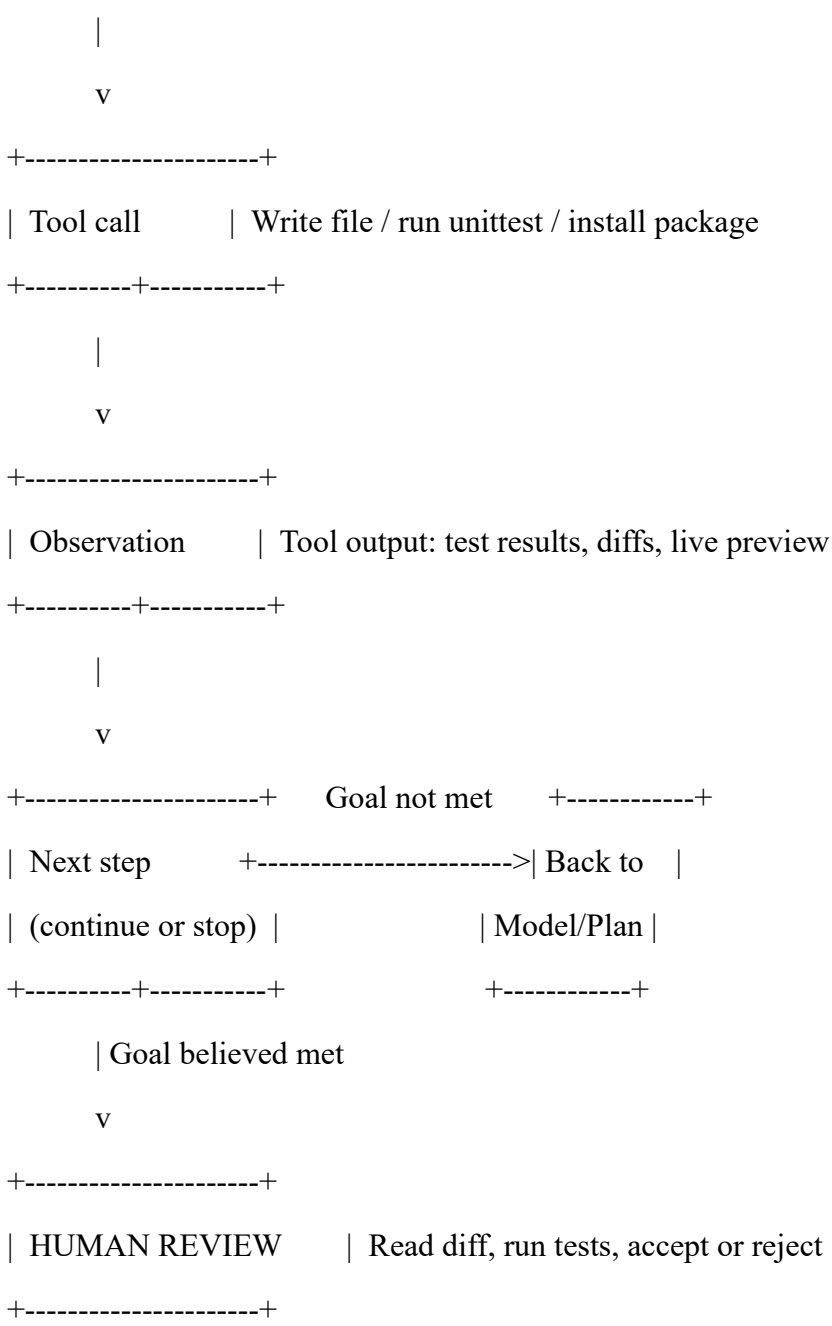
Issue	Reproduction	Priority	Prompt / Change	Retest Result
2. Web Prototype (Functional / AC1): Decimal & non-integer input handling	Entering 630.5 in HTML <input type="number" > in v1 was permitted by default browser behavior without strict integer string validation.	Medium-High (AC1 requires integer minutes)	Targeted prompt in Bolt.new: validate raw input strings with strict integer parsing (Number.isInteger) before evaluating AC1–AC4 and display an inline AC1 error message.	Retested with 630.5, empty string, and 660: decimals/empty strings are rejected with clear AC1 feedback; 660 is accepted.
3. Web Prototype (Usability & Mobile / AC6): Lack of HH:MM labels & cramped <400px layout	In v1, users had to convert raw minutes (540, 660) to clock time (09:00, 11:00) mentally, and mobile viewports (375px) had cramped inputs without quick boundary presets.	Medium (AC6 usability & mobile layout)	Targeted prompt in Bolt.new: display live HH:MM clock equivalents beside each minute field, stack inputs vertically below 400px, and add non-mutating Quick Test Preset buttons.	Verified at 375px width: HH:MM labels update live, no horizontal scroll occurs, and all AC1–AC6 checks pass.

Task 6: Coding-Agent Analysis

1. Agent Loop Diagram

Plaintext





2. Context, Tools, and Permissions

- **Claude (AI Assistant with Sandbox):**

  Context: Conversation history, explicit AC1–AC6 prompt constraints, and session files (booking.py, test_booking.py).

  Tools: Sandbox file creation/editing, shell execution (python -m unittest, diff), and file download export.

  Permissions: Isolated cloud container only; zero access to the user's local filesystem or operating system.

- **Bolt.new (Browser-Based Coding Agent):**

  Context: User prompts plus the project's full file tree inside the browser workspace.

Tools: File read/write, terminal commands, npm package installation, Vite dev server, and live DOM preview.

Permissions: Full read/write/execute access inside the in-browser WebContainer sandbox, isolated from the host OS but permitted to fetch external npm packages.

3. Observed Behavior vs. Internal Assumptions

Observed Behavior: Staged plan generation, creation of booking.py v1 and v2, execution of 50 unit tests, diff outputs showing v1 raising TypeError/ValueError on malformed existing, and live file-tree/preview updates in Bolt.new.

Assumptions about Internal Implementation: The exact contents of hidden system prompts, internal step-orchestration or retry logic, and potential model routing behind the platform interfaces.

4. Safety Checkpoint

Diff-and-Test Gate Before Accepting Changes: Before accepting any AI-generated code from Claude or Bolt.new, we inspect the diff, check package.json for unrequested dependencies, and execute the test suite independently.

Reliability: Running dynamic boundary probes caught the [(True, 5)] interval bug in v1 that static reading missed.

Security: Reviewing diffs and package.json prevents unreviewed or vulnerable third-party packages from being introduced automatically by the agent.

Human Responsibility: An AI tool cannot be held accountable for production failures; human verification ensures every acceptance criterion is genuinely met.

Task 7: Reflection - Dependability and AI (156 Words)

Our contract-first Python workflow (plan, generate, critique, repair) produced a small, testable function whose behavior was fixed by AC1 to AC5 before any code existed, whereas Bolt.new's full-page vibe coding delivered a working interface quickly but made UI and input decisions we had to refine. Each approach carried distinct risks: v1 of booking.py accepted the malformed interval [(True, 5)] and raised exceptions on None, a reliability fault; raw minute fields without HH:MM conversion and cramped layouts below 400px created an availability and usability risk; and unreviewed package installation by a browser-based agent is a security risk. These cases show why code review, tool permissions, and testing remain human responsibilities: a model cannot be accountable for what it ships, and only our tests exposed its gaps. In v2, we deliberately rejected adding range checks (0 <= s < e <= 1440) on existing intervals to avoid scope creep. Next, without expanding scope, we would add automated ARIA accessibility checks and property-based tests for can_book